Identifying Microservices Using Functional Decomposition

Shmuel Tyszberowicz[1], Robert Heinrich[2], Bo Liu[3,4](✉), and Zhiming Liu[3]

[1] The Academic College Tel-Aviv Yafo, Tel Aviv, Israel
tyshbe@mta.ac.il
[2] Karlsruhe Institute of Technology, Karlsruhe, Germany
heinrich@kit.edu
[3] Key Laboratory of Safety-Critical Software (Nanjing University of Aeronautics and Astronautics), Ministry of Industry and Information Technology, Nanjing, China
[4] Southwest University, Chongqing, China
{liubocq,zhimingliu88}@swu.edu.cn

Abstract. The microservices architectural style is rising fast, and many companies use this style to structure their systems. A big challenge in designing this architecture is to find an appropriate partition of the system into microservices. Microservices are usually designed intuitively, based on the experience of the designers. We describe a systematic approach to identify microservices in early design phase which is based on the specification of the system's functional requirements and that uses functional decomposition. To evaluate our approach, we have compared microservices implementations by three independent teams to the decomposition provided by our approach. The evaluation results show that our decomposition is comparable to manual design, yet within a much shorter time frame.

Keywords: Microservices · Decomposition · Coupling · Cohesion Clustering

1 Introduction

The microservices architecture style is rising fast as it has many advantages over other architectural styles such as scalability (fine-grained, independently scalable), improved fault isolation (and thus resilience), and enhanced performance. Hence, many companies are using this architectural style to develop their systems; for example, Netflix, Amazon, eBay, and Uber. The microservices architecture is an approach for developing an application as a set of small, well-defined, cohesive, loosely coupled, independently deployable, and distributed services. Microservices interact via messages, using standard data formats and protocols and published interfaces using a well-defined lightweight mechanism such as REST [9]. An important aspect of this architecture is that each microservice owns its domain model (data, logic, and behavior). Related functionalities are

© Springer Nature Switzerland AG 2018
X. Feng et al. (Eds.): SETTA 2018, LNCS 10998, pp. 50–65, 2018.
https://doi.org/10.1007/978-3-319-99933-3_4

combined into a single business capability (called bounded context), and each microservice implements one such capability (one or several services) [24]. The microservices architecture assists in tackling the complexity of large applications by decomposing them into small pieces, where each component resides within its own bounded context.

This architecture also enables traceability between the requirements and the system structure, and thus only one microservice has to be changed and redeployed in order to update a domain [10].

The development of the microservices architecture aims to overcome shortcomings of *monolithic* architecture styles, where the user interface, the business logic, and the databases are packaged into a single application and deployed to a server. Whereas deployment is easy, a large monolithic application can be difficult to understand and to maintain, because once the system evolves, its modularity is eroded. Besides, every change causes the redeployment of the whole system.

Two important issues which are favored by the microservices community as keys to building a successful microservices architecture are the functional decomposition of an application and the decentralized governance (i.e., each service usually manages its unique database). One of the big challenges in designing the microservices architecture is to find an appropriate partition of the system into microservices [28]. For example, the microservice architecture can significantly affects the performance of the system [20]. It seems reasonable that each service will have only a very limited set of responsibilities, preferable only one—the single responsibility principle [27]. Determining the microservice granularity influences the quality of service of the microservice application [16] and also the number of microservices. Nevertheless, there is a lack of systematic approaches that decide and suggest the microservice boundaries as described in more detail in the following section. Hence, microservices design is usually performed intuitively, based on the experience of the system designers. However, providing an inappropriate partition into services and getting service boundaries wrong can be costly [29].

In this paper we describe a systematic approach to identify microservices in early design phase. We identify the relationships between the required system operations and the state variables that those operations read or write. We then visualize the relationships between the system operations and state variables, thus we can recognize clusters of dense relationships. This provides a partition of the system's state space into microservices, such that the operations of each microservice access only the variables of that microservice. This decomposition guarantees strong cohesion of each microservice and low coupling between services.

The remainder of the paper is organized as follows. The state of the art is discussed in Sect. 2. We use the CoCoME [32] case study to motivate and demonstrate our approach; in Sect. 3 we present the CoCoME trading system. Our approach for identifying microservices is described in Sect. 4. Systems evolve over time, and in Sect. 5 we describe the KAMP approach for change impact analysis which we will use for system maintenance. In Sect. 6, we evaluate our approach. We conclude in Sect. 7.

2 State of the Art

We now present the state of the art on identifying microservices. In the approach proposed by Newman [29], bounded contexts (i.e., responsibility have explicit boundaries) play a vital role in defining loosely coupled and high cohesive services. However, the question about how to systematically find those contexts remains open.

Use-cases are mentioned in [28] as an obvious approach to identify the services. Others, e.g. [14], suggest a partition strategy based on verbs. Some approaches for partitioning a system into microservices are described in [34]. Those approaches include: using nouns or resources, by defining a service that is responsible for all operations on entities or resources of a given type; decomposing by verbs or use cases and define services that are responsible for particular actions; decomposing by business capability, where a business capability is something that a business does in order to generate value; and decomposing by domain-driven design subdomain, where the business consists of multiple subdomains, each corresponding to a different part of the business. The domain-driven design approach [10] seems to be the most common technique for modeling microservices.

Some of the approaches listed in [34] are relevant to our approach (e.g., using the use cases, nouns, verbs). However, no systematic approach is offered. Baresi et al. [2] propose an automated process for finding an adequate granularity and cohesiveness of microservices candidates. This approach is based on the semantic similarity of foreseen/available functionality described through OpenAPI specifications (OpenAPI is a language-agnostic, machine-readable interface for REST APIs). By leveraging a reference vocabulary, this approach identifies potential candidate microservices, as fine-grained groups of cohesive operations (and associated resources). A systematic architectural modeling and analysis for managing the granularity of the microservices and deciding on the boundaries of the microservices is provided in [16]. The authors claim that reasoning about microservice granularity at the architectural level can facilitate analysis of systems that exhibit heterogeneity and decentralized governance.

However, there hardly are any guidelines on what is a 'good' size of a microservice [13,29]. Basically the suggestion is to refine too large services, without providing a metric that defines what too large means. Practical experience shows that the size of the microservices heavily differ from one system to another. There is also no rigorous analysis of the actual dependencies between the system's functionality and its structure.

3 Running Example: CoCoME

To demonstrate our approach we have applied the CoCoME (Common Component Modeling Example) case study on software architecture modeling [19,32]. It represents a trading system as can be observed in a supermarket chain handling sales, and is widely used as a common case study for software architecture

modeling and evolution [18]. The example includes processing sales at a single store of the chain, e.g. scanning products or paying, as well as enterprise-wide administrative tasks, e.g. inventory management and reporting.

The system specification includes functional requirements for: selling products, ordering products from providers, receiving the ordered products, showing reports, and managing the stocks in each store. The specification is informal, and is given in terms of detailed use cases (in the format proposed by Cockburn [5]). In the following, we provide an excerpt of the use cases of CoCoME, as depicted in Fig. 1. A fully detailed description can be found in [32].

- *Process Sale*: this use case detects the products that a customer has purchased and handles the payment (by credit or cash) at the cash desk.
- *Order Products*: this use case is employed to order products from suppliers.
- *Receive Ordered Products*: this use case describes the requirement that once the products arrive at the store, their correctness have to be checked and the inventory has to be updated.
- *Show Stock Reports*: this use case refers to the requirement of generating stock-related reports.
- *Show Delivery Reports*: calculate the mean times a delivery takes.
- *Change Price*: describes the case where the sale price of a product is changed.

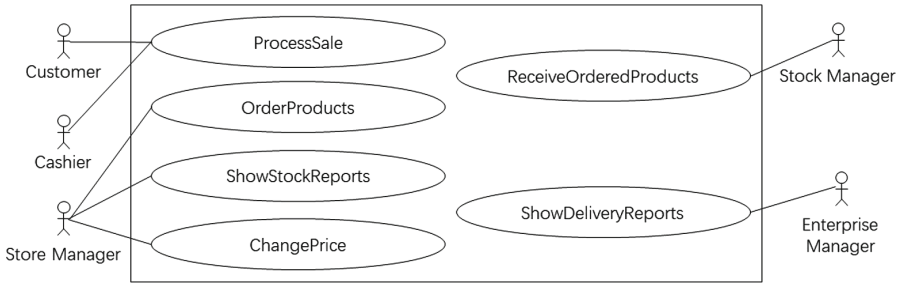

Fig. 1. The UML use case diagram for the CoCoME system.

4 Identifying Microservices

The proposed analytical approach to identify microservices described in this section is based on use case specification of the software requirements and on a functional decomposition of those requirements. To employ the suggested approach, we first need to create a model of the system. This model consists of a finite set of *system operations* and of the system's *state space*. System operations are the public operations (methods) of the system; i.e., the operations that comprise the system's API and which provide the system response to external triggers. The state space is the set of system variables which contain the information that system operations write and read.

System Decomposition. The decomposition of the system into microservices is achieved by partitioning the state space in a way that guarantees that each microservice has its own state space and operations. That is, the microservices partition the system state variables into disjoint sets such that the operations of each microservice may directly access only its local variables. When a microservice needs to refer (read or write) to a variable in another state space, it is achieved only via the API of the relevant microservice, i.e., the one that includes the relevant state space. This enables the selection of a good decomposition—i.e., one that guarantees low coupling as well as high cohesion. Thus, we model a system decomposition into microservices as a syntactical partition of the state space. A system is then built as a composition of microservices by conjoining their operations.

System requirements can be given in many forms, ranging from informal natural language to fully formal models. Even an existing implementation of the system (e.g., a monolith one) can serve as a source of requirements (see, e.g., [7], [12]). Use case specifications are widely accepted as a way of specifying functional requirements [21]. A use case describes how users (actors) employ a system to achieve a particular goal. We identify the *system operations* and the *system state variables* based on the description of the use cases. We record—in what we call *operation/relation table*—the relationships between each system operation and the state variables that the operation used (reads or writes). That is, each cell in the table indicates whether the operation writes to the state variable, reads it, or neither writes to it nor reads it. In order to identify the system operations and system state variables, we find—as a first approximation[1]—all the verbs in the informal descriptions of the use cases. The nouns found in those descriptions serve as an approximation for the system state variables [1][2]. Note that our approach works in general once the operations and state variables are identified, without the need to know how they are gathered. Nevertheless, we shortly describe how we have collected the information that is used to create the operation/relation table, as it makes the process even more systematic, compared to any ad-hoc approach of extracting variables and operations. We use tools (e.g., easyCRC [31], TextAnalysisOnline [39]) that assist us to extract nouns and noun phrases from the use case specifications (as candidates for state variables) as well as verbs (suggesting system operations). This process, however, can be done without using any tool. This systematic approach enables us to identify operations based on the use case descriptions and to produce informal and formal specification of the contracts of the system operations. This then allows us to carry out formal analysis and validation of the system model [25], as discussed in Sect. 6.

Visualization. The operation/relation table is then visualized, shown in a graph form. The visualization as a graph enables us to identify clusters of dense

[1] The list of verbs that we have found may be updated as some verbs may not be system operations, others may not be mentioned in the informal description, and some verbs are synonyms, hence they describe the same operation.

[2] A brain storming is needed to handle issues such as synonyms, irrelevant nouns, etc.

relationships that are weakly connected to other clusters.[3] Each such cluster is considered a good candidate to become a microservice, because:

1. The amount of information it shares with the rest of the system is small, thus it is protected from changes in the rest of the system and vice versa—this satisfies the low coupling requirement.
2. The internal relationships are relatively dense, which in most cases indicates a cohesive unit of functionality, satisfying the demand for strong cohesion.

We build an undirected bipartite graph G whose vertices represent the system's state variables and operations. An edge connects an operation *op* to a state variable v if and only if *op* either reads the value of v or updates it. In addition, we assign a weight to each edge of G, depending on the nature of the connection. A read connection has a lower weight (we have chosen 1) and a write connection has a higher weight (in our case 2). This choice tends to cluster together data with those operations that change it, preferring read interfaces between clusters. A write interface actively engages both subsystems, thus it has a stronger coupling. While different numbers are possible for the weights, the chosen numbers result in a graph that satisfies our needs to identify clearly separated clusters. Note, however, that we have also tried other weights—yet keeping the weight of the write operation higher than that of the read operation. Whereas this sometimes has changed the layout of the graph, it did not change the clustering. For the visualization of the graph we use `NEATO` [30]—a spring model based drawing algorithm. The program draws undirected graphs such that nodes that are close to each other in graph-theoretic space (i.e. shortest path) are drawn closer to each other. The left hand side of Fig. 2 presents an example of the operation/relation dependency graph of the CoCoME system, as drawn by `NEATO`. Note that this visualization can be used in various ways: to suggest low dependency partitions, where each part can serve as a microservice; to evaluate partitions into microservices that are dictated by non-functional constraints; and to explore changes to the system model that reduce the dependencies between areas that we consider as good microservices candidates. We have used the partition into clusters to identify the possible microservices. The right hand side of Fig. 2 describes the microservices that we have identified.

At this point it is important to emphasize that the idea of software clustering is not a new one; the reader may refer, for example, to [8,26]. Those works investigate clustering based on source code analysis, and the main idea is to enable developers to understand the structure of evolving software. The source level modules and dependencies are mapped to a module dependency graph and then the graph is partitioned so that closely related nodes are grouped into composite nodes—the clusters (subsystems). However, who guarantees that the design of the developed system was 'good' with respect to strong cohesiveness and weak coupling? Quoting [26]: "Creating a good mental model of the structure of a complex system, and keeping that mental model consistent with changes

[3] A clustering of a graph G consists of a partition of the node set of G.

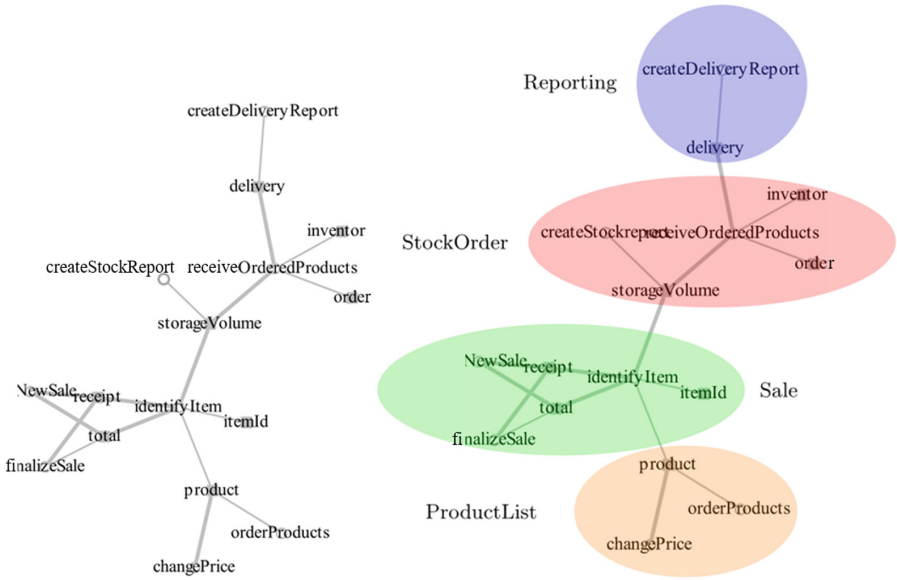

Fig. 2. An operation/relation dependency graph of CoCoME. The left side shows the diagram before identifying the microservices, and the right side presents also the microservices (the colored shapes). Thin/thick edges represent read/write relationship; circles represent operations; and squares represent state variables. Note that the graph was created by NEATO; the truncation in names (e.g. order) was done by NEATO. (Color figure online)

that occur as the system evolves, is one of many serious problems that confront today's software developers. . . . we have developed an automatic technique to decompose the structure of software systems into meaningful subsystems. Subsystems provide developers with high-level structural information that helps them navigate through the numerous software components, their interfaces, and their interconnections".

Finding the clusters at the source code level definitely helps to understand the structure of the system. But it may be the case that the design is bad with regard to coupling and cohesion. Correcting this once the code exists is sometimes a very difficult task. Using clustering before the code exists, as done in our approach, enables to develop software that is of higher design quality. It is also easier than to maintain the software, for example with tools like KAMP, as elaborated in Sect. 5. Moreover, having both the operation/relation table and the clustering that has been obtained using this table, traceability is much easier. Suppose, for example, that the user has to change a function; then she can easily recognize in which component this function is implemented.

APIs and Databases. For each identified microservice we define its API and its data storage. The API of the microservice (cluster) is provided by the union

of the system operations which write into the state variables belonging to the cluster that has been identified based on the visualization process. When a system operation of another cluster reads information from the current cluster, a getter method is added to the API of the current cluster; i.e., the access is only through a published service interface. For example, the operation *identifyItem* is part of the API of the *Sale* microservice (see the right hand side of Fig. 2). Since *identifyItem* needs to read the *product* state variable of the *ProductList* microservice, the *getProduct* operation is added to *ProductList*'s API.

There are two basic approaches regarding using databases for microservices. (i) Hasselbring and Steinacker [17] propose the *share nothing* approach according to which each microservice should have its own database. The advantages of this approach is higher speed, horizontal scalability, and improved fault-tolerance. One can also use a polyglot persistence architecture, where for each service the best suited type of database is chosen. However, this approach is at the price of data consistency, since consistency may only be eventual consistency (see [37]), and problems are dealt with by compensating operations. (ii) Yanaga [38] as well as Lewis and Fowler [24] claim that information can be shared. Yanaga argues that since a microservice is not an island, the data between services has to be integrated. That is, a microservice may require information provided by other services and provides information required by other services.

We agree that services sometimes need to share data. Nevertheless, this sharing should be minimal, to make the microservices as loosely coupled as possible. In our approach we analyze each created cluster. The information that needs to be persistent is found in the various clusters. If needed, we add to the cluster (i.e., microservice) a database that contains the persistent data that is private to this specific service. Of course it might be that the persistent data is located in different clusters. In this case we may end up with several microservices that contain data that is needed by other services. We guarantee that those databases are accessible only via the API of the services that contain them; i.e., no direct database access is allowed from outside the service.

Approach Summary. Our approach can be summarized as follows:

1. Analyze the use case specifications (write out their detailed descriptions if needed).
2. Identify the system operations and the system state variables (based on the use cases and their scenarios).
3. Create an *operation/relation* table.
4. Advise a possible decomposition into high cohesive and low coupled components (using a visualization tool).
5. Identify the microservices APIs.
6. Identify the microservices databases.
7. Implement the microservices (using RESTish protocols as the means of communication between microservices).
8. Deploy.

We have not referred to implementation and deployment; this is done in Sect. 6.

5 Architecture-Based Change Impact Analysis

Software systems must evolve over time, since otherwise they progressively become less useful [23]. Once a system is released, it continuously changes, e.g. due to emerging user requirements (perfective changes), bug fixes (corrective changes), platform alterations (adaptive changes), or correction of latent faults before they become operational faults (preventive changes). Consequently, the system drifts away from its initial architecture due to the evolutionary changes; yet, knowledge about the software architecture is essential to predict maintenance efforts.

The KAMP approach [35] supports software architects assessing the effect of change requests on technical and organizational work areas during software evolution. Based on the Palladio Component Model (for details see [33]), KAMP supports modeling the initial software architecture—the *base architecture*, and the architecture after a certain change request has been implemented in the model—the *target architecture*. The KAMP tooling calculates the differences between the base and the target architecture models and analyses the propagation of changes in the software system. Due to this change propagation analysis, services affected by a given change request can be identified. A large number of affected services may indicate the necessity for redesigning the system. In such a case we have to update the operation table according to the new requirements, and to continue in the process as described in this paper.

6 Evaluation

In this section, we exemplify our approach based on the CoCoME community case study [32] and evaluate our results.

Case Study: Starting with the use case specification of CoCoME, we identify the system operations as well as their state variables. Table 1 shows the operation/relation table that has been created based on this information. This table is then visualized as a graph, shown on the left hand side of Fig. 2. Based on the graphical representation we have identified four major clusters which are candidates to become microservices: *ProductList*, *Sale*, *StockOrder*, and *Reporting*; see the right hand side of Fig. 2. In this way we have achieved a meaningful partition into clusters, where each cluster has high cohesion and the coupling between the clusters is low. Each of the four identified microservice candidates (clusters) is responsible for a single bounded context in a business domain [10]. As can be seen on the right hand side of Fig. 2, the microservices are not totally independent. The communication between the microservices is implemented using REST APIs [29].

Note that the emphasize in our approach is on identifying microservices that deal with one business capability, rather than minimizing the microservices size; this conforms to [6]. Moreover, as stated in [6], a clear cohesion based on a high number of dependencies indicates the need for a single microservice. This

Table 1. Operation/relation table for CoCoME

Operation	itemId	mode	receipt	product	total	storageVolume	order	creditDetails	inventory	delivery	suppliers
startNewSale			2		2						
identifyItem	1		2	1	2	2					
finalizeSale			2		1						
cardPayment								2			
changeMode		2									
orderProducts				1						2	1
receiveOrderedProducts						2	1		2	2	
createStockReport						1					
createDeliveryReport				1						1	
changePrice				2							

is exactly what our functional decomposition achieves, and it actually provides bounded context—as suggested in the domain-driven design approach [10].

Evaluation Goal and Design: To evaluate our approach, we have compared our proposed decomposition of the CoCoME system into microservices with the microservices that have been identified by three independent software projects that have independently implemented CoCoME; the implementers have not been aware of the other implementations. They also have not known our approach of system decomposition. One project has been developed in RISE Centre at Southwest University (SWU-RISE) in Chongqing, China, and the other two have been built in Karlsruhe Institute of Technology (KIT), Germany. All projects have been developed by students. The supervisors of the projects have not been involved in the microservices' design nor did they provide any hint that might have influenced the design. The goal of the evaluation is to check whether our approach provides a decomposition of the system into microservices that is similar to a decomposition suggested by humans. Of course it might be the case that the latter decomposition is wrong; therefore we compared the results to three implementations. Note that there may exist several decompositions of a given system, where each has its advantages and disadvantages. Thus, we cannot claim that the decomposition provided by our approach is *the* best one, while other decompositions are not as good. If our approach provides results that are comparable to the decompositions done by human developers, then it has the advantage that due to its systematic and tool-supported nature it provides the decomposition much faster than when done manually.

Two of the CoCoMe implementations have been developed in KIT. Two master students, working independently of each other, have decomposed the

system into microservices as part of the requirements in a practical course on software engineering. The students have basic knowledge in software architecture styles and in developing microservices. The students decomposed the system into microservices merely based on the existing use case specification and on an existing source code of a service-oriented implementation of CoCoME. To identify the microservices, the students also had to understand the design documentation—a component diagram and sequence diagrams [19]. They have identified microservices candidates based on the domain-driven design [10]. Then they modeled the application domain and divided it into different bounded contexts, where each bounded context was a candidate to be a microservice. Later they have made explicit the interrelationships between the bounded contexts. After understanding the requirements, it was a matter of days for them to design the microservices architecture of the system. The design and implementations created by the students can be found on GitHub.[4]

Another group of students, in the RISE Centre at Southwest University (SWU-RISE), has also been involved in the evaluation of our approach. This team has been composed of three computer science students: two first year post graduate students and one undergraduate (senior student). The students have basic knowledge in software architecture styles and in developing microservices. All students have a basic idea of SOA, web services, and microservices-based systems. One postgraduate student also has more than one year experience in web-based system development and Docker usage. The team had a supervisor that worked with them to control the progress of the development and to provide consulting work on requirements (thus acting as the software client). The students have been responsible for all development phases: requirements analyzing and modeling, system design, and system implementation and deployment. The supervisor, however, was not involved in the actual design of the microservice, and as mentioned provided no hints that may have led to the design proposed in this paper. The following principles guided this team in their division into microservices:

- *Identify business domain objects.* For example, the CoCoME domain objects includes *Order*, *Commodity*, *Transaction*, *Payment*, etc. The aim of this principle is to find those microservices that correspond to one domain object. This process was done by analyzing the use case descriptions and building a conceptual class model [3].
- *Identify special business.* This is the case that a business process spans multiple domain objects, and then an independent microservice is designed.
- *Reuse.* Business processes that are frequently called may be detached from the object and designed as a single microservice. Accordingly, if a constructed microservice is seldom used or difficult to be implemented separately, it can be attached to some object microservice or other microservices.

The team used the rCOS approach [4] to analyze the requirements and design the microservices. The process can be summarized as follows: identify the use

[4] For the implementations see https://github.com/cocome-community-case-study.

cases (this step was not needed, as brief descriptions of the use cases specifications have been provided); construct the conceptual class diagram and use case sequence diagrams; for each use case operation write contracts in terms of its pre- and post-conditions. Those artifacts have formed a formal requirements model; analysis can then be applied to verify consistency and correctness [4]. The CoCoMe requirements have been analyzed and validated for consistency and functional correctness using the AutoPA tool [25]. It was a matter of days for the students to design the microservices architecture of the system.[5]

Evaluation Results: Although the students at KIT named the microservices differently from the names used by us, both implementations also created four microservices. Each microservice has the same functionality as the one that was provided by employing our approach:

- *ProductList*: Managing the products that are stored in the trading system.
- *Sale*: Handling a sale in the trading system.
- *StockOrder*: Managing a stock order of products.
- *Reporting*: Creating a report of the delivered products.

The microservices identified by the students at KIT are all connected to a Frontend-Service, which provides the basic panel in which the user interfaces of the microservices are displayed. Thus, the Frontend-Service serves as a single integration point, similar to controllers that are used in related approaches. Controllers are responsible for receiving and handling system operations [22]. The use case controller pattern, for example, deals with all system events of a use case. It suggests delegating the responsibility for managing a use case flow of execution to a specialized controller object, thus localizing this functionality and its possible changes. The controllers provide a uniform way of coordinating actor events, user interfaces, and system services.

The microservices that have been identified by the students at KIT and their relationship are depicted in Fig. 3.

The students at SWU-RISE identified eight microservices:

- *Inventory*: Handles the products.
- *Commodity*: Queries the information regarding the product to be sold.
- *Order*: Manages the orders.
- *Supplier*: Handles the suppliers.
- *Transaction*: Refers to the sale (creation, management, query and end of the transactions).
- *Supplier_evaluation*: Calculates average time for the supplier to deliver the product to the supermarket.
- *Inventory_report*: Produces reports.
- *Pay*: Recording payment records—cash or non-cash.

[5] The implementation created by this group of students can be found in http://cocome.swu-rise.net.cn/.

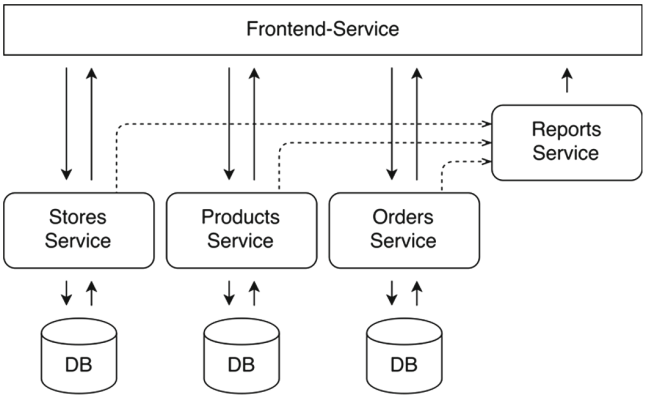

Fig. 3. Overview of the CoCoME microservices designed by KIT students [36]

The microservices that have been identified by the students at SWU-RISE and their relationship are depicted in Fig. 4. As can be seen, there are four different GUIs for the actors of CoCoME. The GUI combines a few microservices together to accomplish a specific function through a gateway.

The evaluation results show that the two teams in KIT have implemented the CoCoME model using the same decomposition as advised by our approach; the only difference is in the names of the microservices. The SWU-RISE team created eight microservices. However, a thorough examination reveals that the extra microservices are a refinement of the microservices provided by our approach. That is, each of the additional implemented microservices also appears as an activity in the coarser suggested decomposition. For example, the *Stock-Order* microservice refers to the variables *order* and *inventory* which became fine-grained microservices in the implementation of the students in SWU-RISE. Also the *Reporting* microservice has been split—into the *Supplier* and *Supplier_evaluation* microservices. This means that our approach can serve as the base decomposition. As mentioned in Sect. 4, the user can explore the visualization and decide whether to further break the system into finer microservices. Doing this results in exactly the decomposition provided by the SWU-RISE group, just using different microservices names. Moreover, recall that our goal is to provide a systematic and meaningful, high cohesive decomposition into microservices rather than finding the finest granular decomposition. A high cohesive cluster that is identified using our approach indicates the need for a microservice, and this microservice sometimes can be refined further. As recognized also in e.g. [6,15], a design problem of developing a system is to find the optimal level of granularity of a microservice architecture, and it is not always an immediate process. Balancing the size and the number of microservices is a design trade-off.

Our approach guarantees that each service can get what it needs from the other services. In contrast, when developing microservices without such a

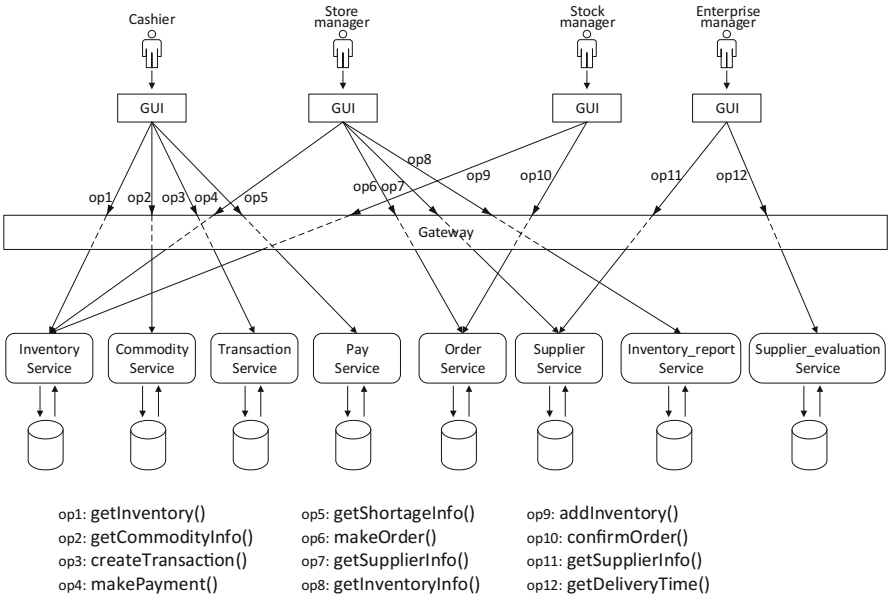

Fig. 4. Overview of the CoCoME microservices designed by SWU-RISE students

systematic approach, it is often difficult to understand and follow the interconnections between the services so thoroughly [29]. The evaluation results give us reasons to believe that our approach identifies microservice in a quality that is comparable to a design done by human software designers. Our approach, however, achieved the microservices identification much faster and with less effort compared to human developers. While identifying the microservices was a matter of days for the students at KIT and SWU-RISE, by employing our approach it was a matter of hours. Moreover, the problem of identifying the appropriate microservices becomes much more complicated as the system becomes more complex. A large real world system has much more details, so the chances of getting an appropriate decomposition by intuition will decrease.

7 Conclusion

We proposed a systematic and practical engineering approach to identify microservices which is based on use-case specification and functional decomposition of those requirements. This approach provides high cohesive and low coupled decomposition. To evaluate our approach, we have compared the results to three independent implementations of the CoCoME system. The evaluation give us reasons to believe in the potential of our approach. We will involve other kind of systems in the evaluation. Doing this we will also examine the scalability of our approach. We believe it is scalable, since the tools that create the diagrams and suggest decompositions are quite fast and can work on large graphs [11].

Moreover, if indeed as in the CoCoME model each use case is handled in only one component provided in the decomposition, then the work can be split among distributed teams.

Acknowledgment. This work was supported by the DFG (German Research Foundation) under the Priority Programme SPP1593, and the Key Laboratory of Safety-Critical Software (Nanjing University of Aeronautics and Astronautics) under the Open Foundation with No. NJ20170007. We would like to thank Kang Cheng, Guohang Guo, Yukun Zhang, Nils Sommer, and Stephan Engelmann who worked on the development of the various CoCoME systems. We also thank the anonymous reviewers for their careful reading and their many insightful comments and suggestions. Their comments helped to improve and clarify this paper.

References

1. Abbott, R.J.: Program design by informal english descriptions. Commun. ACM **26**(11), 882–894 (1983)
2. Baresi, L., Garriga, M., De Renzis, A.: Microservices identification through interface analysis. In: De Paoli, F., Schulte, S., Broch Johnsen, E. (eds.) ESOCC 2017. LNCS, vol. 10465, pp. 19–33. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-67262-5_2
3. Chen, X., He, J., Liu, Z., Zhan, N.: A model of component-based programming. In: Arbab, F., Sirjani, M. (eds.) FSEN 2007. LNCS, vol. 4767, pp. 191–206. Springer, Heidelberg (2007). https://doi.org/10.1007/978-3-540-75698-9_13
4. Chen, Z., et al.: Refinement and verification in component-based model-driven design. Sci. Comput. Program. **74**(4), 168–196 (2009)
5. Cockburn, A.: Writing Effective Use Cases. Addison-Wesley, Boston (2000)
6. de la Torre, C., et al.: .NET Microservices: Architecture for Containerized .NET Applications. Microsoft (2017)
7. De Santis, S., et al.: Evolve the Monolith to Microservices with Java and Node. IBM Redbooks, Armonk (2016)
8. Doval, D., Mancoridis, S., Mitchell, B.S.: Automatic clustering of software systems using a genetic algorithm. In: STEP, pp. 73–81. IEEE (1999)
9. Dragoni, N., et al.: Microservices: yesterday, today, and tomorrow. Present and Ulterior Software Engineering, pp. 195–216. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-67425-4_12
10. Evans, E.: Domain Driven Design: Tackling Complexity in the Heart of Business Software. Addison-Wesley, Boston (2004)
11. Faitelson, D., Tyszberowicz, S.: Improving design decomposition. Form. Asp. Comput. **22**(1), 5:1–5:38 (2017)
12. Fowler, M.: MonolithFirst (2015). https://martinfowler.com/bliki/MonolithFirst.html#footnote-typical-monolith. Accessed Mar 2018
13. Francesco, P.D., et al.: Research on architecting microservices: trends, focus, and potential for industrial adoption. In: ICSA, pp. 21–30. IEEE (2017)
14. Hassan, M., Zhao, W., Yang, J.: Provisioning web services from resource constrained mobile devices. In: IEEE CLOUD, pp. 490–497 (2010)
15. Hassan, S., Bahsoon, R.: Microservices and their design trade-offs: a self-adaptive roadmap. In: SCC, pp. 813–818. IEEE (2016)

16. Hassan, S., et al.: Microservice ambients: an architectural meta-modelling approach for microservice granularity. In: ICSA, pp. 1–10. IEEE (2017)
17. Hasselbring, W., Steinacker, G.: Microservice architectures for scalability, agility and reliability in E-Commerce. In: ICSA Workshops, pp. 243–246. IEEE (2017)
18. Heinrich, R., et al.: A platform for empirical research on information system evolution. In: SEKE, pp. 415–420 (2015)
19. Heinrich, R., et al.: The CoCoME platform for collaborative empirical research on information system evolution. Technical report 2016:2, KIT, Germany (2016)
20. Heinrich, R., et al.: Performance engineering for microservices: research challenges and directions. In: Companion Proceedings of ICPE, pp. 223–226 (2017)
21. Jacobson, I., et al.: Object-Oriented Software Engineering - A Use Case Driven Approach. Addison-Wesley, Boston (1992)
22. Larman, C.: Applying UML and Patterns, 3rd edn. Prentice Hall, Upper Saddle River (2004)
23. Lehman, M.M.: On understanding laws, evolution, and conservation in the large-program life cycle. J. Syst. Softw. **1**, 213–221 (1980)
24. Lewis, J., Fowler, M.: Microservices. https://martinfowler.com/articles/microservices.html. Accessed Apr 2018
25. Li, X., Liu, Z., Schäf, M., Yin, L.: AutoPA: automatic prototyping from requirements. In: Margaria, T., Steffen, B. (eds.) ISoLA 2010. LNCS, vol. 6415, pp. 609–624. Springer, Heidelberg (2010). https://doi.org/10.1007/978-3-642-16558-0_49
26. Mancoridis, S., et al.: Bunch: a clustering tool for the recovery and maintenance of software system structures. In: ICSM, pp. 50–59. IEEE Computer Society (1999)
27. Martin, R.C.: Agile Software Development: Principles, Patterns, and Practices. Prentice Hall, Upper Saddle River (2003)
28. Namiot, D., Sneps-Sneppe, M.: On micro-services architecture. J. Open Inf. Technol. **2**(9), 24–27 (2014)
29. Newman, S.: Building Microservices. O'Reilly, Sebastopol (2015)
30. North, S.C.: Drawing graphs with NEATO. User Manual (2004)
31. Raman, A., Tyszberowicz, S.S.: The EasyCRC tool. In: ICSEA, pp. 52–57. IEEE (2007)
32. Rausch, A., Reussner, R., Mirandola, R., Plášil, F. (eds.): The Common Component Modeling Example: Comparing Software Component Models. LNCS, vol. 5153. Springer, Heidelberg (2008). https://doi.org/10.1007/978-3-540-85289-6
33. Reussner, R.H., et al.: Modeling and Simulating Software Architectures - The Palladio Approach. MIT Press, Cambridge (2016)
34. Richardson, C.: Microservices from design to deployment (2016). https://www.nginx.com/blog/microservices-from-design-to-deployment-ebook-nginx/
35. Rostami, K., Stammel, J., Heinrich, R., Reussner, R.: Architecture-based assessment and planning of change requests. In: QoSA, pp. 21–30 (2015)
36. Sommer, N.: Erweiterung und Wartung einer Cloud-basierten JEE-Architektur (in German), report of a practical course. Technical report, KIT, Germany (2017)
37. Vogels, W.: Eventually consistent. Commun. ACM **52**(1), 40–44 (2009)
38. Yanaga, E.: Migrating to Microservice Databases: From Relational Monolith to Distributed Data. O'Reilly, Sebastopol (2017). E-book
39. Text analysis. http://textanalysisonline.com/textblob-noun-phrase-extraction. Accessed Apr 2018